This experiment demonstrated core Docker essentials including Dockerfile creation, image optimization using .dockerignore, tagging strategies, multi-stage builds, and image publishing. These concepts are fundamental in modern DevOps workflows for building efficient, secure, and portable containerized applications.

Experiment 5: Docker - Volumes, Environment Variables, Monitoring & Networks

Part 1: Docker Volumes - Persistent Data Storage

Lab 1: Understanding Data Persistence

The Problem: Container Data is Ephemeral

```
# Create a container that writes data docker run
-it --name test-container ubuntu /bin/bash
# Inside container:
echo "Hello World" > /data/message.txt
cat /data/message.txt # Shows "Hello
World" exit

# Restart container docker start test-
container docker exec test-container cat
/data/message.txt
# ERROR: File doesn't exist! Data was lost.
Solution: Docker Volumes
```

```
PS C:\Users\HP\Desktop\exp 5> docker run -it --name test-container ubuntu ba
sh
root@83f53322ba3b:/# mkdir /data
echo "Hello World" > /data/message.txt
cat /data/message.txt
exit
Hello World
exit
PS C:\Users\HP\Desktop\exp 5> docker rm -f test-container
test-container
PS C:\Users\HP\Desktop\exp 5> docker run --rm ubuntu cat /data/message.txt
cat: /data/message.txt: No such file or directory
```

Lab 2: Volume Types

```
# Create anonymous volume (auto-generated name)
docker run -d -v /app/data --name web1 nginx
```

```
# Check volume
docker volume ls
# Inspect container to see volume mount
docker inspect web1 | grep -A 5 Mounts

2. Named Volumes
Bash
# Create named volume
docker volume create
mydata

# Use named volume docker run -d -v
mydata:/app/data --name web2 nginx

# List volumes
docker volume
ls # Shows:
mydata

# Inspect volume docker
volume inspect mydata
```

```
PS C:\Users\HP\Desktop> docker volume ls
DRIVER    VOLUME NAME
local     839780c6bfe8acb10e6753236c354bd32b20cd2f6b7cc1065cc8759b8424bdfa
local     mydata
local     mysql-data
local     pgdata
local     postgres-data
local     redis-data
PS C:\Users\HP\Desktop> docker inspect mydata
[
    {
        "CreatedAt": "2026-02-18T03:26:28Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
        "Name": "mydata",
        "Options": null,
        "Scope": "local"
    }
]
```

1. Bind Mounts (Host Directory) ```Bash

Create directory on host

mkdir ~/myapp-data

Mount host directory to container

docker run -d -v ~/myapp-data:/app/data –name web3 nginx

Add file on host

echo “From Host” > ~/myapp-data/host-file.txt

Check in container

docker exec web3 cat /app/data/host-file.txt

Shows: From Host

```
![alt text](/Containerisation-and-devops-LAB-EXPERIMENTS/Lab/lab5/image-2.png)

### Lab 3: Practical Volume Examples
Example 1: Database with Persistent
Storage

```Bash
# MySQL with named volume
docker run -d \
 --name mysql-db \
 -v
mysql-data:/var/lib/mysql \  -
e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0

# Check data persists
docker stop mysql-db
docker rm mysql-db

# New container with same volume
docker run -d \
 --name new-mysql \
 -v
mysql-data:/var/lib/mysql \ -e
MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
# Data is preserved!
```

Example 2: Web App with Configuration Files
```

```
# Create config directory
mkdir ~/nginx-config

#  Create  nginx  config
file     echo     'server
{ listen 80;
 server_name localhost;  location / {
 return 200 "Hello from mounted config!";
 }
}' > ~/nginx-config/nginx.conf

# Run nginx with config bind mount
docker run -d \
 --name nginx-custom \
 -p 8080:80 \
 -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
nginx

# Test
curl http://localhost:8080
```

```
PS C:\Users\HP\Desktop\exp 5> docker run -d --name mysql-db -v mysql-data:/var/lib/mysql -e
 MYSQL_ROOT_PASSWORD=secret mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
bde62e757594: Pull complete
e7a04c019bde: Pull complete
4f37333d1be6: Pull complete
a9a9deeee02a: Pull complete
e05db5310ebc: Pull complete
d442b2c1726e: Pull complete
2e2c1f6f8d57: Pull complete
23fbf4028535: Pull complete
f508d7fab5b3: Pull complete
ce98f3559366: Pull complete
bae900376130: Pull complete
e6e53730409b: Download complete
8393005deaac: Download complete
Digest: sha256:99d774bf02a48a1bb1c599920d2571946d31e5940b854b02737d5e95c184358f
Status: Downloaded newer image for mysql:8.0
d438c86c2cb6ffa0541f25e2d3de0f3294ca6520b5bec0e952cff6e96ff2f3c9
PS C:\Users\HP\Desktop\exp 5> # In PowerShell, create a local folder and file
PS C:\Users\HP\Desktop\exp 5> mkdir myapp-data


    Directory: C:\Users\HP\Desktop\exp 5


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         2/18/2026   9:00 AM                myapp-data


PS C:\Users\HP\Desktop\exp 5> echo "From Host" > myapp-data/host-file.txt
PS C:\Users\HP\Desktop\exp 5>
PS C:\Users\HP\Desktop\exp 5> # Mount it
PS C:\Users\HP\Desktop\exp 5> docker run -d -v ${PWD}/myapp-data:/app/data --name web3 ngin
x
235afc29b992cfaba7adda5e825236871305d96aaee3414861a6ab3cfeba3e95
PS C:\Users\HP\Desktop\exp 5>
PS C:\Users\HP\Desktop\exp 5> # Verify
PS C:\Users\HP\Desktop\exp 5> docker exec web3 cat /app/data/host-file.txt
```

Lab 4: Volume Management Commands

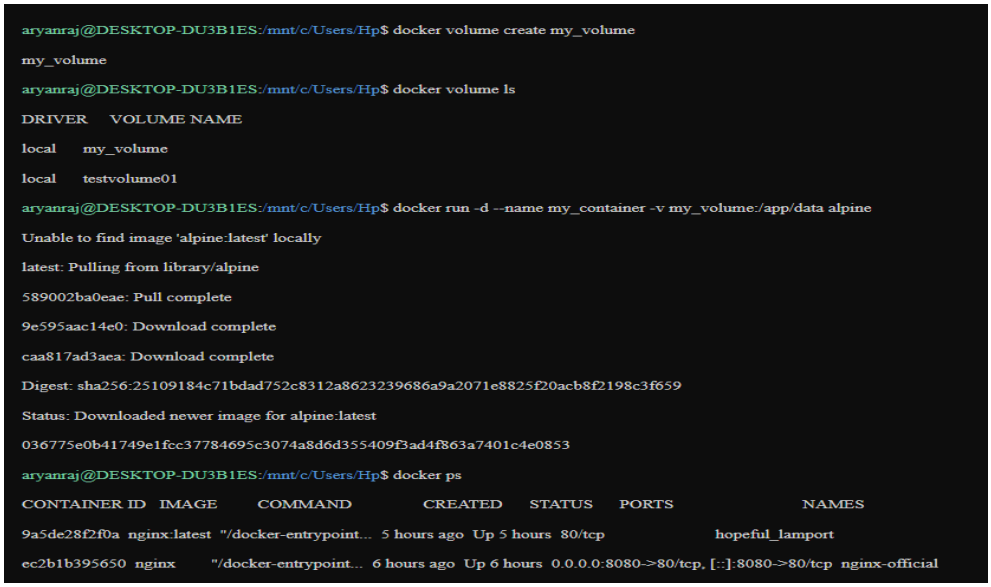
```
# List all volumes
docker volume ls
# Create a volume docker
volume create app-volume

# Inspect volume details
docker volume inspect app-
volume

# Remove unused volumes
docker volume prune

# Remove specific volume
docker volume rm volume-
name

# Copy files to/from volume
docker cp local-file.txt container-name:/path/in/volume
```



Part 2: Environment Variables

Lab 1: Setting Environment Variables

Method 1: Using -e flag

```
# Single variable
docker run -d \
 --name app1 \
 -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
 -e DEBUG="true" \
-p 3000:3000 \
my-node-app
```

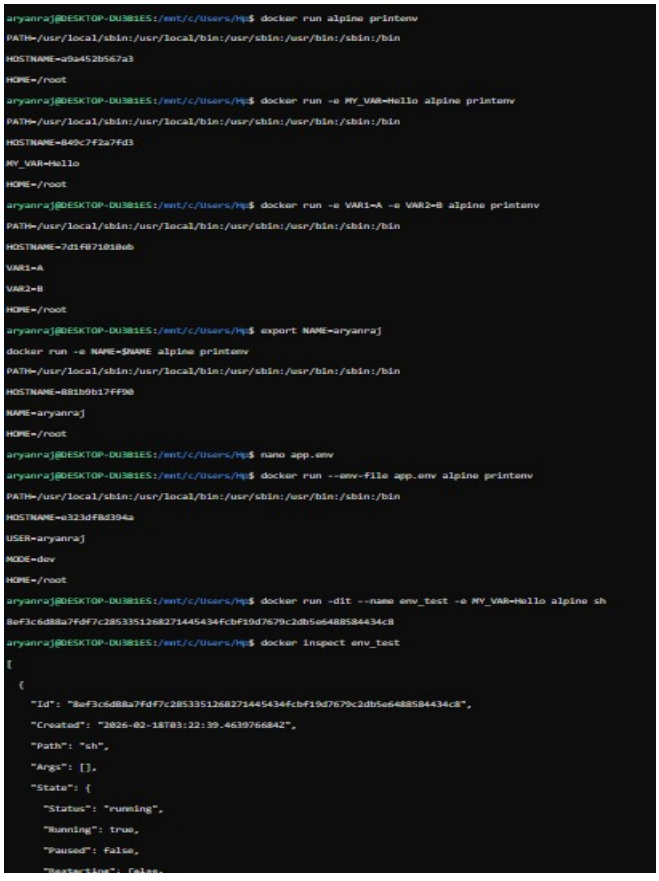
```
# Multiple
variables docker
run -d \  -e
VAR1=value1 \  e
VAR2=value2 \  -e
VAR3=value3 \
myapp
```

Method 2: Using –env-file

```
# Create .env file echo
"DATABASE_HOST=localhost" > .env
echo "DATABASE_PORT=5432" >> .env
echo "API_KEY=secret123" >> .env

# Use env file
docker run -d \
--env-file .env \
--name app2 \ my-
app

# Use multiple env files
docker run -d \
 --env-file .env \ --env-
file .env.secrets \ my-app
```



Method 3: In Dockerfile

```
# Set default environment variables
```

```
ENV NODE_ENV=production
ENV PORT=3000
ENV APP_VERSION=1.0.0
# Can be overridden at runtime
```

Lab 2: Environment Variables in Applications

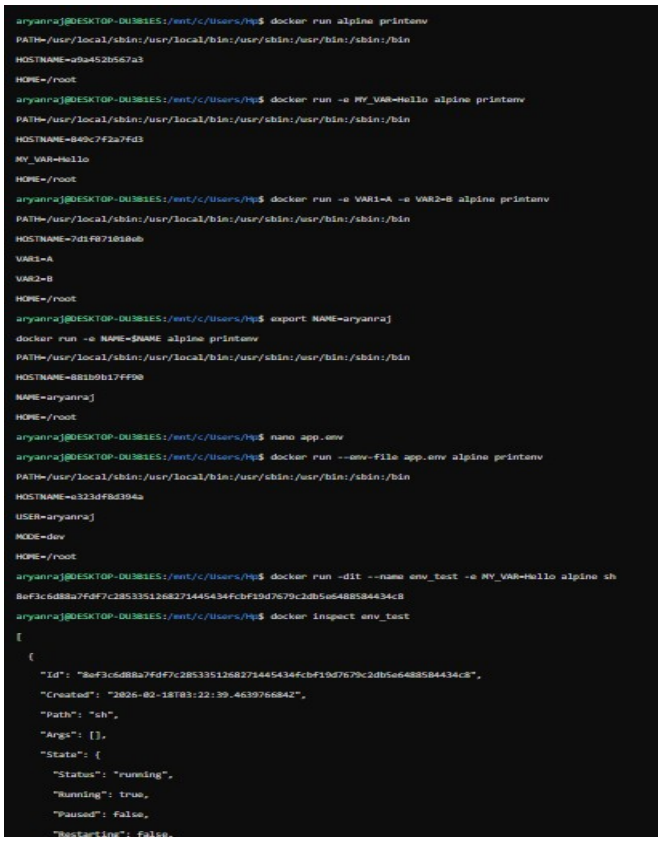
```
Python Flask Example

app.py import os from
flask import Flask

app = Flask(__name__)
Read environment
variables
db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() ==
'true' api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
 port = int(os.environ.get('PORT', 5000))
app.run(host='0.0.0.0', port=port, debug=debug_mode)
```



Dockerfile with Environment Variables

```
Dockerfile
FROM python:3.9-slim
Set environment variables at build time
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

WORKDIR /app COPY
requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .

# Default runtime environment variables
ENV PORT=5000
ENV DEBUG=false

EXPOSE 5000
CMD ["python", "app.py"]
```

```python
import os
from flask import Flask

app = Flask(__name__)

@app.route('/config')
def config():
    return {
        'db_host': os.environ.get('DATABASE_HOST', 'localhost'),
        'debug': os.environ.get('DEBUG', 'false'),
        'port': os.environ.get('PORT', '5000')
    }

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=int(os.environ.get('PORT', 5000)))
```

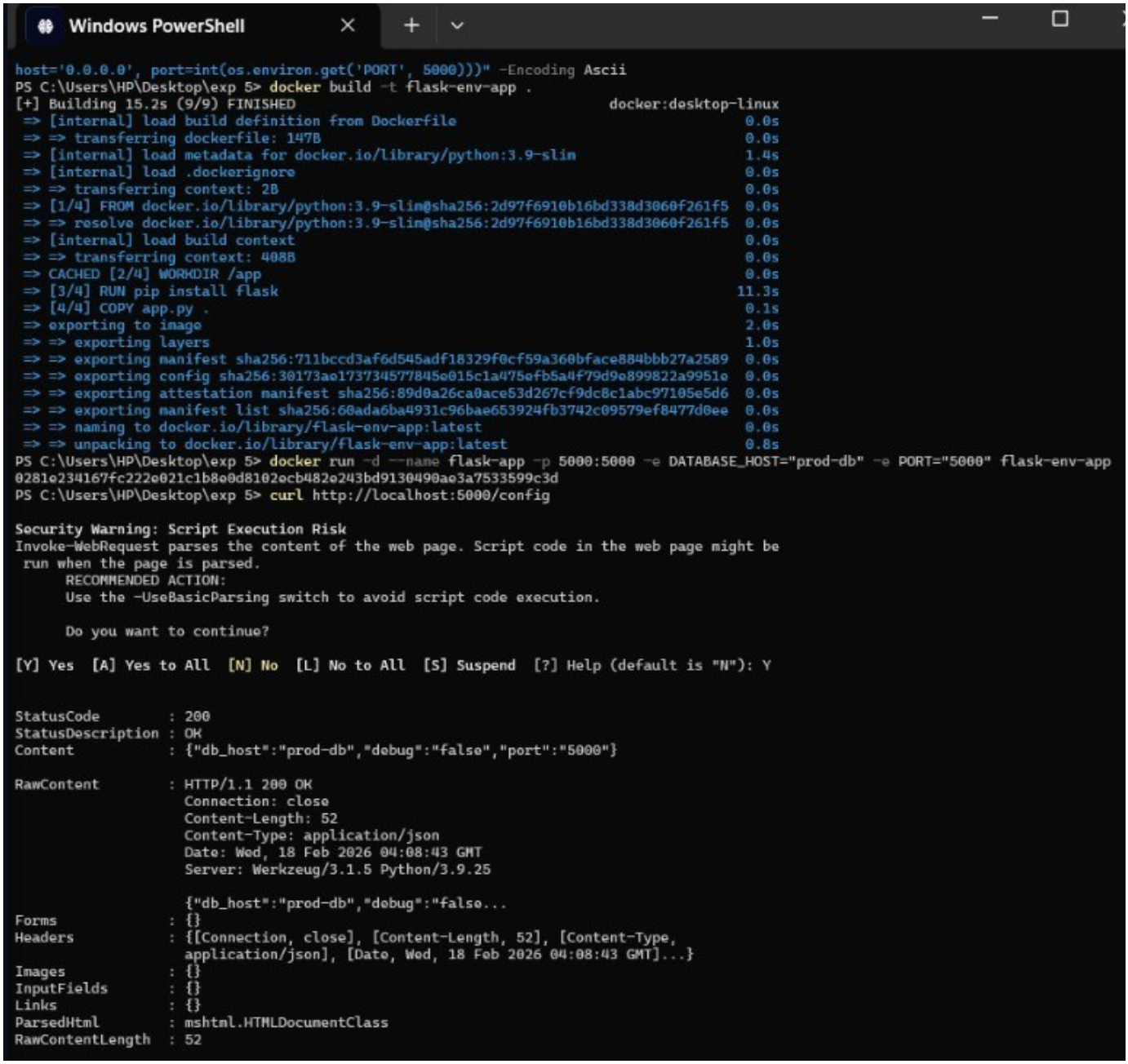
Lab 3: Test Environment Variables

```
# Run with custom env vars

docker run -d \  --
name flask-app \
 -p 5000:5000 \
 -e DATABASE_HOST="prod-db.example.com" \
```

```
  -e DEBUG="true" \
  -e PORT="8080" \
  flask-app
  # Check environment in running container
  docker exec flask-app env docker exec
  flask-app printenv DATABASE_HOST

  # Test the endpoint curl
  http://localhost:5000/config
```



Part 3: Docker Monitoring Lab 1:

Basic Monitoring Commands docker

stats - Real-time Container Metrics

```
  # Live stats for all containers
  docker stats
```

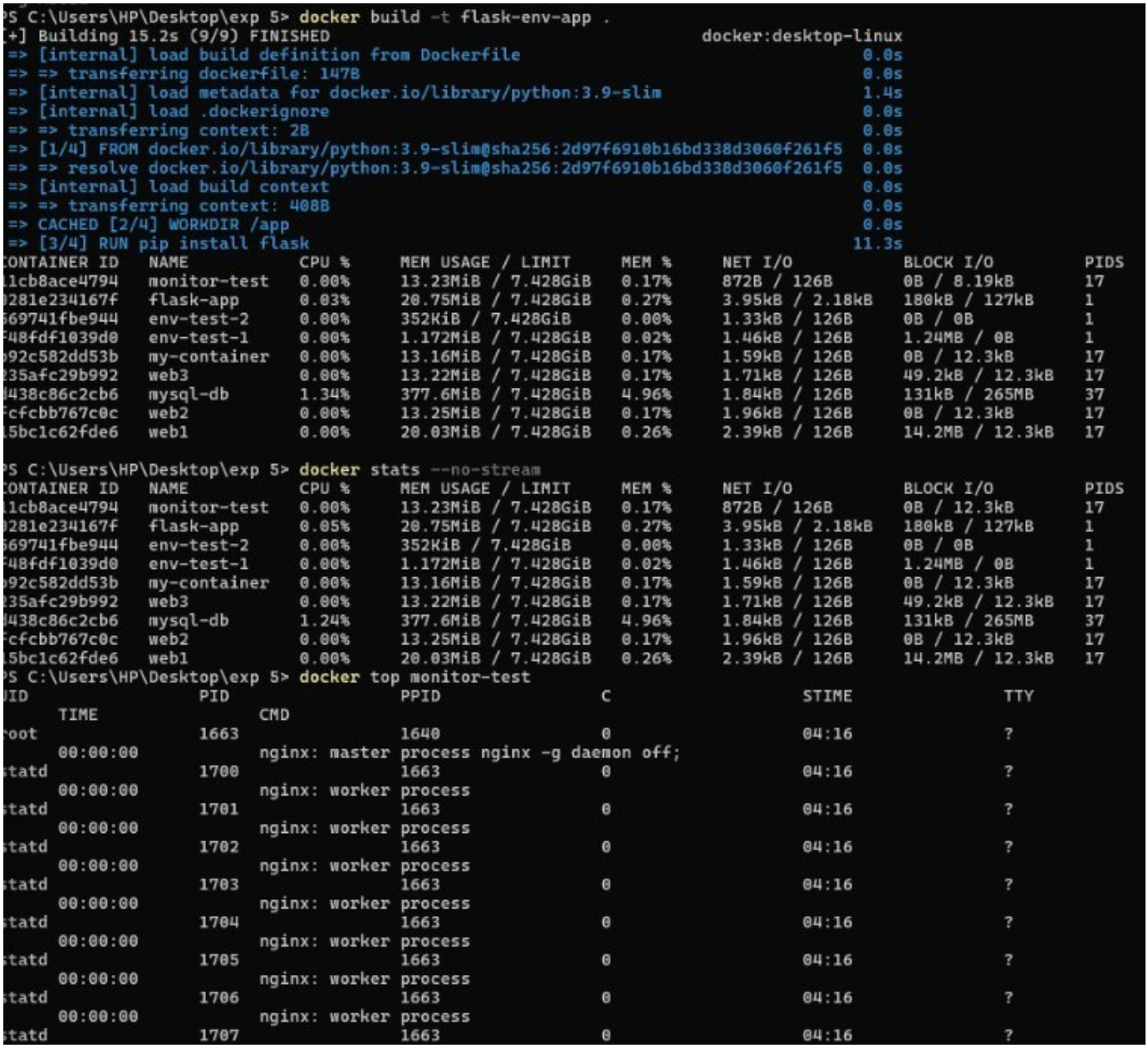
```
# Live stats for specific
containers docker stats
container1 container2

# Specific format output docker
stats --format "table \t\t\t"

# No-stream (single snapshot)
docker stats --no-stream

# All containers (including
stopped) docker stats --all

# Useful Format Options: # Custom format docker
stats --format "Container: | CPU: | Memory: "
# JSON output docker stats --format
json --no-stream
# Wide output docker stats --no-
stream --no-trunc
```



Lab 2: docker top - Process Monitoring

```
# View processes in container
docker top container-name

# View with full command line
docker top container-name -ef

# Compare with host
processes ps aux | grep
docker
```

Lab 3: docker logs - Application Logs

```
# View logs docker logs

container-name # Follow

logs (like tail -f)
```

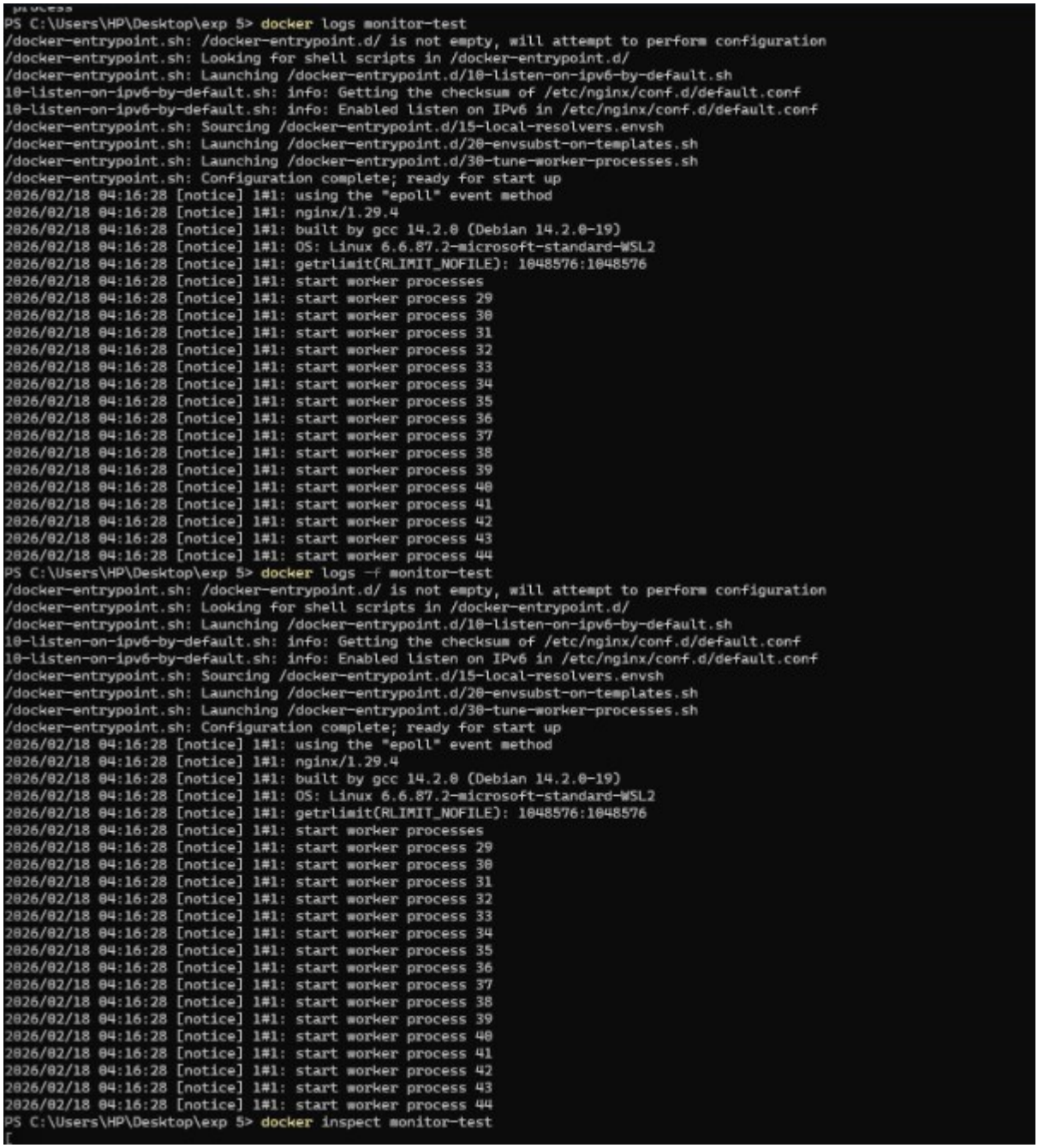
```
docker logs -f container-
name


# Last N lines docker logs --tail
100 container-name

# Logs with timestamps
docker logs -t container-
name

# Logs since specific time docker logs --
since 2024-01-15 container-name

# Combine options docker logs -f --tail
50 -t container-name
```



Lab 4: Container Inspection

```
# Detailed container info
docker inspect container-
name

# Specific information docker inspect --
format='' container-name docker inspect
--format=''    container-name    docker
inspect --format='' container-name

# Resource limits
docker inspect --format='' container-name
docker inspect --format='' container-name
Lab 5: Events Monitoring
Bash
# Monitor Docker events in real-time docker
events

# Filter events docker events --
filter 'type=container' docker
events --filter 'event=start' docker
events --filter 'event=die'

# Since specific time docker
events --since '2024-01-15'

# Format output docker
events --format ' '
```

Lab 6: Practical Monitoring Script

```
#!/bin/bash
# monitor.sh
# Simple Docker monitoring

echo "=== Docker Monitoring Dashboard
===" echo "Time: $(date)" echo

echo "1. Running Containers:"
docker ps --format "table \t\t"
echo

echo "2. Resource Usage:"
docker stats --no-stream --format "table \t\t\t"
echo

echo "3. Recent Events:" docker events --since '5m' --
until '0s' --format ' ' |tail -5 echo
echo "4. System Info:"
docker system df
```

Part 4: Docker Networks

Lab 1: Understanding Docker Network Types

```
# Default networks
docker network ls

# Output:
# NETWORK ID         NAME              DRIVER            SCOP
# abc123             bridge            bridge            E
# def456             host              host              loca
# ghi789             none              null              l
                                                         loca
                                                         l
                                                         loca
                                                         l
```

Lab 2: Network Types
Explained 1. Bridge Network
(Default)
Bash

```
# Containers on bridge network can communicate
# Each container gets own IP, isolated from host

# Create custom bridge network
docker network create my-network

# Inspect network docker
network inspect my-network

# Run containers on custom network docker run -d --
name web1 --network my-network nginx docker run -d
--name web2 --network my-network nginx

# Containers can communicate using container names
docker exec web1 curl http://web2
```

```
PS C:\Users\HP\Desktop> docker network ls
NETWORK ID     NAME              DRIVER    SCOPE
2f47d48207ed   bridge            bridge    local
e8a2ffd50282   docker_gwbridge   bridge    local
01c62e85d65b   host              host      local
jutru9wvzfh9   ingress           overlay   swarm
vpqupqja99mi   my-overlay        overlay   swarm
cfe3e5982d1f   my_bridge         bridge    local
1a9bdc5c06fb   my_macvlan        macvlan   local
94242e3b4ca3   none              null      local
PS C:\Users\HP\Desktop> docker network create my-network
31265cf2bc5fa35dd6f96c17ad2e97c6af2ccc6d01cdde6ab48007de2b8b3e15
PS C:\Users\HP\Desktop> docker network inspect my-network
[
    {
        "Name": "my-network",
        "Id": "31265cf2bc5fa35dd6f96c17ad2e97c6af2ccc6d01cdde6ab48007de2b8b3e15",
        "Created": "2026-02-24T15:00:13.127990858Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.20.0.0/16",
                    "Gateway": "172.20.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {},
        "Containers": {},
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.20.0.0/16": {
                        "IPsInUse": 3,
                        "DynamicIPsAvailable": 65533
                    }
```

2.Host Network

```
# Container uses host's network directly # No
network isolation, shares host's IP docker run -
d --name host-app --network host nginx

# Access directly on host port 80
curl http://localhost
```

```
PS C:\Users\HP\Desktop> docker run -d --name host-app --network host nginx
a8a6ff84ed57fc397730a302c93f4d0eb1be6f6ecef916fc09fab4d2aa80d85a
PS C:\Users\HP\Desktop> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS     NAMES
a8a6ff84ed57   nginx     "/docker-entrypoint.…"   43 seconds ago   Up 42 seconds             host-app
acaa9def39d7   nginx     "/docker-entrypoint.…"   3 minutes ago    Up 3 minutes    80/tcp    web2
e02130a64606   nginx     "/docker-entrypoint.…"   4 minutes ago    Up 4 minutes    80/tcp    web1
PS C:\Users\HP\Desktop> docker exec host-app curl http://localhost
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
```

3.None Network

```
    # No network access
docker run -d --name isolated-app --network none alpine sleep 3600

    # Test no network interfaces
    docker exec isolated-app
    ifconfig
```

```
PS C:\Users\HP\Desktop> docker run -d --name isolated-app --network none alpine sleep 3600
354f3f134be991481b23e9a84a5a9d526804f4e536e44f0c9562d1fbe826ba6a
PS C:\Users\HP\Desktop> docker exec isolated-app ifconfig
lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

4.Overlay Network (Swarm)

```
    # For Docker Swarm multi-host networking
    docker network create --driver overlay my-
    overlay
```

```
NETWORK ID     NAME              DRIVER    SCOPE
4f9e2aa7454d   app-network       bridge    local
2f47d48207ed   bridge            bridge    local
e8a2ffd50282   docker_gwbridge   bridge    local
01c62e85d65b   host              host      local
jutru9wvzfh9   ingress           overlay   swarm
31265cf2bc5f   my-network        bridge    local
c82c625e08a5   myapp-network     bridge    local
94242e3b4ca3   none              null      local
PS C:\Users\HP\Desktop>
```

Lab 3: Network Management Commands

```
# Create network docker network create app-network docker network
create --driver bridge --subnet 172.20.0.1/16 my-subnet

# Connect container to network docker network
connect app-network existing-container

# Disconnect container from network docker
network disconnect app-network container-name

# Remove network docker
network rm network-name

# Prune unused networks
docker network prune
```

```
Windows PowerShell
PS C:\Users\HP\Desktop> docker network create --driver bridge --subnet 172.30.0.0/16 --gateway 172.30.
0.1 my-subnet
2580cd335bca803ce6a82830345177b021437cac0ad77c8ca1f33d54e9b65e75
PS C:\Users\HP\Desktop> docker network connect app-network web1
PS C:\Users\HP\Desktop> docker network ls
NETWORK ID     NAME              DRIVER    SCOPE
e431ae4cea77   app-network       bridge    local
2f47d48207ed   bridge            bridge    local
e8a2ffd50282   docker_gwbridge   bridge    local
01c62e85d65b   host              host      local
jutru9wvzfh9   ingress           overlay   swarm
31265cf2bc5f   my-network        bridge    local
vpqupqja99mi   my-overlay        overlay   swarm
2580cd335bca   my-subnet         bridge    local
cfe3e5982d1f   my_bridge         bridge    local
1a9bdc5c06fb   my_macvlan        macvlan   local
94242e3b4ca3   none              null      local
PS C:\Users\HP\Desktop> docker network disconnect app-network web1
PS C:\Users\HP\Desktop> docker network rm network-name
Error response from daemon: network network-name not found
exit status 1
PS C:\Users\HP\Desktop> docker network rm app-network
app-network
PS C:\Users\HP\Desktop> docker network prune
WARNING! This will remove all custom networks not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Networks:
my-overlay
my_macvlan
my_bridge
my-subnet
```

Lab 4: Multi-Container Application Example

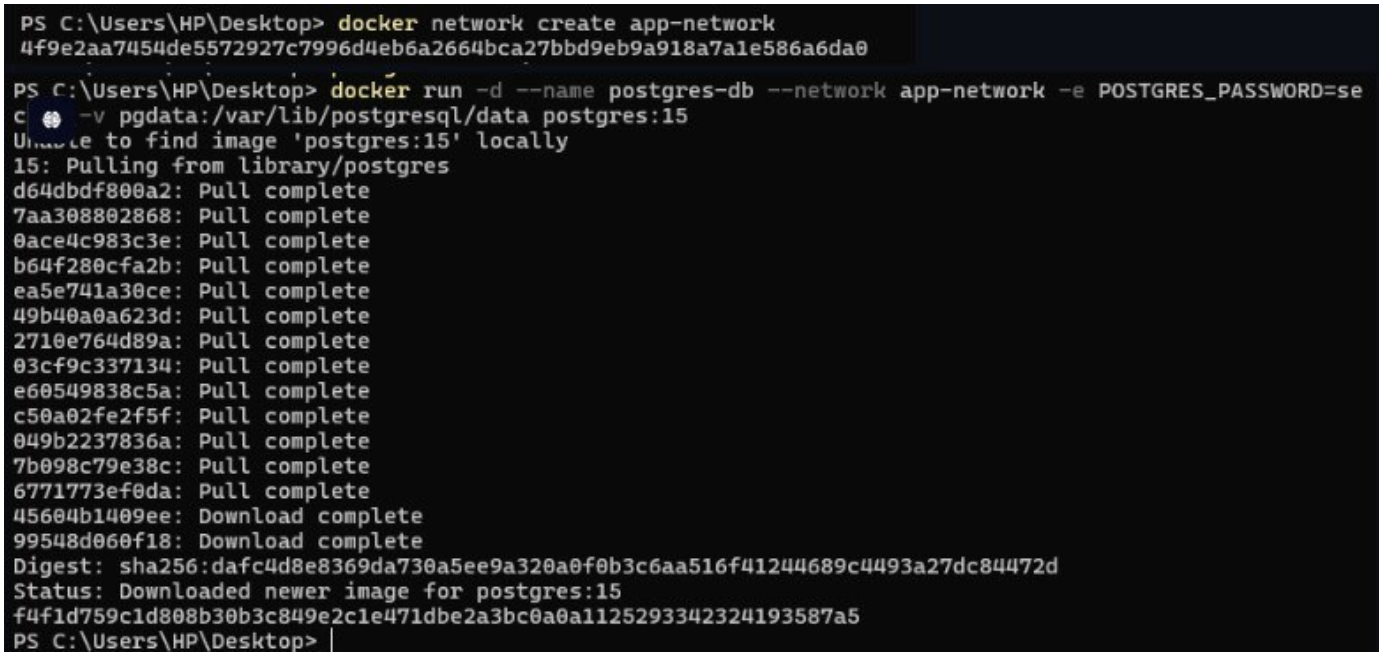
Web App + Database Communication

```
# Create network docker
network create app-network

# Start database
docker run -d \  --
name postgres-db \
 --network app-network \
 -e POSTGRES_PASSWORD=secret \
 -v pgdata:/var/lib/postgresql/data \
postgres:15


# Start web application
docker run -d \
 --name web-app \
 --network app-network \
 -p 8080:3000 \
 -e DATABASE_URL="postgres://postgres:secret@postgres-db:5432/mydb" \
 -e DATABASE_HOST="postgres-db" \
node-app


# Web app can connect to database using "postgres-db" hostname
```



Lab 5: Network Inspection & Debugging

```
# Inspect network docker
network inspect bridge

# Check container IP
docker inspect --format='' container-name

# DNS resolution test
docker exec container-name nslookup another-container

# Network connectivity test docker exec container-name
ping -c 4 google.com docker exec container-name curl -
I http://another-container
```

```
# View network ports
docker port container-
name
```

```
PS C:\Users\HP\Desktop> docker inspect --format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' postgres-db

172.18.0.2
PS C:\Users\HP\Desktop> docker port web-app
8000/tcp -> 0.0.0.0:8080
8000/tcp -> [::]:8080
```

Lab 6: Port Publishing vs Exposing

```
# PORT PUBLISHING (host:container)
docker run -d -p 80:8080 --name app1
nginx # Host port 80 → Container port
8080

# Dynamic port publishing
docker run -d -p 8080 --name app2
nginx # Docker assigns random host
port

# Multiple ports
docker run -d -p 80:80 -p 443:443 --name app3 nginx

# Specific host IP
docker run -d -p 127.0.0.1:8080:80 --name app4 nginx

# EXPOSE in Dockerfile (metadata only)
# Dockerfile: EXPOSE 80
# Still need -p to publish
```

```
PS C:\Users\HP\Desktop> #lab 6 of 4 part
PS C:\Users\HP\Desktop> docker run -d -p 80:8080 --name app1 nginx
8b49c19526962c2268d5aa204fc2441949d2a34428c8ad874b7604631088e882
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpo
int app1 (2aec79fefb950dae6dca3043b8ce321ba96db166bbb076286effb4eb682a02f5): failed to bind host port 0.0.0.0:80/tcp: address alrea
dy in use

Run 'docker run --help' for more information
PS C:\Users\HP\Desktop> docker run -d -p 8081 --name app2 nginx
d7c0ed69f5da628af7c7ef817ed57707238a39362001389bdc476bb2af67bb2b
PS C:\Users\HP\Desktop> docker port app2
8081/tcp -> 0.0.0.0:50167
8081/tcp -> [::]:50167
PS C:\Users\HP\Desktop> docker run -d -p 80:80 -p 443:443 --name app3 nginx
35a0ad34e5a8dc4b44a84feb958e2b53d584f69b23e8d1c5a6c64b247b29dbf8
```

Part 5: Complete Real-World Example

Application Architecture:

Flask Web App (port 5000)

PostgreSQL Database (port 5432)

Redis Cache (port 6379)

All connected via custom network

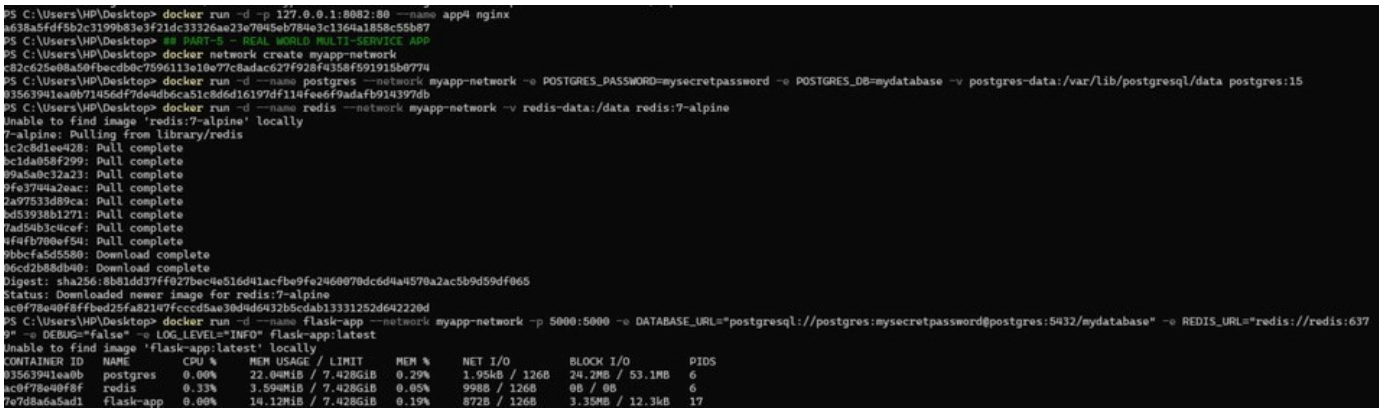
Implementation:

```
# 1. Create network docker
network create myapp-network

# 2. Start database with volume
docker run -d \
 --name postgres \
 --network myapp-network \
 -e POSTGRES_PASSWORD=mysecretpassword \
 -e POSTGRES_DB=mydatabase \
 -v postgres-data:/var/lib/postgresql/data \
postgres:15

# 3. Start Redis
docker run -d \  --
name redis \
 --network myapp-network \
-v redis-data:/data \
redis:7-alpine

# 4. Start Flask app with all configurations
docker run -d \
 --name flask-app \
 --network myapp-network \
 -p 5000:5000 \
 -v $(pwd)/app:/app \
 -v app-logs:/var/log/app \
 -e DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/
mydatabase" \
 -e REDIS_URL="redis://redis:6379" \
 -e DEBUG="false" \
 -e LOG_LEVEL="INFO" \  --
env-file .env.production \
flask-app:latest
```




```
Monitoring commands
# Check all
components docker ps

# Monitor resources docker stats
postgres redis flask-app

# Check logs docker
logs -f flask-app
```

```
# Network connectivity test docker
exec flask-app ping -c 2 postgres
docker exec flask-app ping -c 2 redis

# View network details docker
network inspect myapp-network
```

```
PS C:\Users\HP\Desktop> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED       STATUS       PORTS                                                                                                NAMES
7e7d8a6a5ad1   nginx          "/docker-entrypoint.…"   3 hours ago   Up 3 hours   0.0.0.0:5000->80/tcp, [::]:5000->80/tcp                                                              flask-app
ac0f78e40f8f   redis:7-alpine "docker-entrypoint.s…"   3 hours ago   Up 3 hours   6379/tcp                                                                                             redis
0356394lea8b   postgres:15    "docker-entrypoint.s…"   3 hours ago   Up 3 hours   5432/tcp                                                                                             postgres
a638a5fdf5b2   nginx          "/docker-entrypoint.…"   3 hours ago   Up 3 hours   127.0.0.1:8082->80/tcp                                                                               app4
ed41814b03a8   nginx          "/docker-entrypoint.…"   3 hours ago   Up 3 hours   0.0.0.0:8083->80/tcp, [::]:8083->80/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp                   app3
d7c0ed69f5da   nginx          "/docker-entrypoint.…"   3 hours ago   Up 3 hours   0.0.0.0:50167->8081/tcp, [::]:50167->8081/tcp                                                        app2
58b840ed3b73   nginx          "/docker-entrypoint.…"   4 hours ago   Up 4 hours   0.0.0.0:8080->3000/tcp, [::]:8080->3000/tcp                                                          web-app
f4f1d759c1d8   postgres:15    "docker-entrypoint.s…"   4 hours ago   Up 4 hours   5432/tcp                                                                                             postgres-db
a8a6ff84ed57   nginx          "/docker-entrypoint.…"   6 hours ago   Up 6 hours                                                                                                        host-app
acaa9def39d7   nginx          "/docker-entrypoint.…"   6 hours ago   Up 6 hours   80/tcp                                                                                               web2
e02130a64606   nginx          "/docker-entrypoint.…"   6 hours ago   Up 6 hours   80/tcp                                                                                               web1
PS C:\Users\HP\Desktop>
```

```
PS C:\Users\HP\Desktop> docker logs flask-app
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/24 17:23:20 [notice] 1#1: using the "epoll" event method
2026/02/24 17:23:20 [notice] 1#1: nginx/1.29.4
2026/02/24 17:23:20 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/02/24 17:23:20 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/02/24 17:23:20 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/02/24 17:23:20 [notice] 1#1: start worker processes
2026/02/24 17:23:20 [notice] 1#1: start worker process 29
2026/02/24 17:23:20 [notice] 1#1: start worker process 30
2026/02/24 17:23:20 [notice] 1#1: start worker process 31
2026/02/24 17:23:20 [notice] 1#1: start worker process 32
2026/02/24 17:23:20 [notice] 1#1: start worker process 33
2026/02/24 17:23:20 [notice] 1#1: start worker process 34
2026/02/24 17:23:20 [notice] 1#1: start worker process 35
2026/02/24 17:23:20 [notice] 1#1: start worker process 36
2026/02/24 17:23:20 [notice] 1#1: start worker process 37
2026/02/24 17:23:20 [notice] 1#1: start worker process 38
2026/02/24 17:23:20 [notice] 1#1: start worker process 39
2026/02/24 17:23:20 [notice] 1#1: start worker process 40
2026/02/24 17:23:20 [notice] 1#1: start worker process 41
2026/02/24 17:23:20 [notice] 1#1: start worker process 42
2026/02/24 17:23:20 [notice] 1#1: start worker process 43
2026/02/24 17:23:20 [notice] 1#1: start worker process 44
PS C:\Users\HP\Desktop> docker exec flask-app ping -c 2 postgres
OCI runtime exec failed: exec failed: unable to start container process: exec: "ping": executable file not found in $PATH
PS C:\Users\HP\Desktop> docker run --rm --network myapp-network alpine ping -c 2 postgres
PING postgres (172.21.0.2): 56 data bytes
64 bytes from 172.21.0.2: seq=0 ttl=64 time=1.468 ms
64 bytes from 172.21.0.2: seq=1 ttl=64 time=0.261 ms

--- postgres ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.261/0.864/1.468 ms
PS C:\Users\HP\Desktop> docker run --rm --network myapp-network alpine ping -c 2 redisdocker network inspect myapp-network
BusyBox v1.37.0 (2025-12-16 14:19:28 UTC) multi-call binary.

Usage: ping [OPTIONS] HOST

Send ICMP ECHO_REQUESTs to HOST

        -4,-6           Force IP or IPv6 name resolution
        -c CNT          Send only CNT pings
        -s SIZE         Send SIZE data bytes in packets (default 56)
        -i SECS         Interval
        -A              Ping as soon as reply is received
        -t TTL          Set TTL
        -I IFACE/IP     Source interface or IP address
        -W SEC          Seconds to wait for the first response (default 10)
                        (after all -c CNT packets are sent)
        -w SEC          Seconds until ping exits (default:infinite)
                        (can exit earlier with -c CNT)
        -q              Quiet, only display output at start/finish
        -p HEXBYTE      Payload pattern
PS C:\Users\HP\Desktop> docker network inspect myapp-network
[
    {
```

```
PS C:\Users\HP\Desktop> docker run -d --name web-app --network app-network -p 8080:3000 -e DATABASE_URL="postgres://postgres:secret
@postgres-db:5432/mydb" -e DATABASE_HOST="postgres-db" node-app
Unable to find image 'node-app:latest' locally
docker: Error response from daemon: pull access denied for node-app, repository does not exist or may require 'docker login'

Run 'docker run --help' for more information
PS C:\Users\HP\Desktop> docker run -d --name web-app --network app-network -p 8080:3000 nginx
58b840ed3b7353a0d1cb86f093edfdf7acdf63338e58d4ee789ef8e537411955
PS C:\Users\HP\Desktop> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS
  NAMES
58b840ed3b73   nginx          "/docker-entrypoint.…"   7 seconds ago    Up 7 seconds    0.0.0.0:8080->3000/tcp, [::]:8080->3000/tcp
  web-app
f4f1d759c1d8   postgres:15    "docker-entrypoint.s…"   58 minutes ago   Up 58 minutes   5432/tcp
  postgres-db
a8a6ff84ed57   nginx          "/docker-entrypoint.…"   2 hours ago      Up 2 hours
  host-app
acaa9def39d7   nginx          "/docker-entrypoint.…"   2 hours ago      Up 2 hours      80/tcp
  web2
e02130a64606   nginx          "/docker-entrypoint.…"   2 hours ago      Up 2 hours      80/tcp
  web1
PS C:\Users\HP\Desktop> docker exec web-app ping -n 2 postgres-db
OCI runtime exec failed: exec failed: unable to start container process: exec: "ping": executable file not found in $PATH
PS C:\Users\HP\Desktop> docker exec web-app curl http://postgres-db
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
curl: (7) Failed to connect to postgres-db port 80 after 7 ms: Could not connect to server
PS C:\Users\HP\Desktop> docker run --rm --network app-network alpine sh -c "apk add --no-cache postgresql-client >/dev/null 2>&1 &&
 pg_isready -h postgres-db -p 5432"
postgres-db:5432 - accepting connections
```